

Lecture 4 - System Abstraction & "Growing a Language": Case Study Presentations in Week 5

Topic: System Abstraction, Domain Modeling, and Designing for Growth

Format: Each Group: 5-Minute Live Presentation & Discussion

Lecturer's Opening Hook: The "One-Syllable" Challenge (3 mins)

(Before splitting into groups, start the class by reading this exact script to demonstrate Guy Steele's core concept).

Lecturer: "We need to build a new tool for this school. The tool tracks are in the room. A user has a card. The card lets them in. The tool saves time and name. We want the tool to be fast. We want the tool to be safe. Did you notice anything weird about how I just spoke? Every single word I used was exactly **one syllable**. Just like Guy Steele in his famous OOPSLA '98 keynote, I restricted myself to 'primitives'. It sounds childish, right? It takes forever to explain complex ideas this way.

As architects, if we don't design rich, extensible vocabulary (our Domain Model), our developers will be stuck coding in 'one-syllable' primitives forever. Today, we are going to design a system that can *grow*."

The Activity: "Swinburne Smart-Campus Consulting Briefs"

The Core Scenario: Swinburne University wants to replace physical paper attendance sheets with an "Automated Attendance and Access Control System" using swipe cards. Initially, it will track attendance for lectures to reduce manual processing. *However*, the university explicitly stated that at a later stage, the system must grow to handle physical door access to secure facilities, and eventually, library book checkouts.

(Students either form groups or your team of Assignment and select one of the following briefs to present).

Group 1: The Domain Architects (Abstractions)

Scenario: Your job is to define the boundaries of the problem and create the initial abstract map of the university's attendance process before any code is written. **Presentation Prompts:**

1. **Goals:** Identify the primary business goal and at least two secondary objectives of the system described.
2. **Domain Model:** Draw/Describe the core Domain Model for this system (e.g., what are the entities and relationships?).
3. **The Abstraction Link:** How is your domain model an *abstraction* of the physical university? Give an example of a messy "real-world" detail regarding classrooms or students that you purposely left out of your model to manage complexity.

Group 2: The Task Engineers (Implied Workflows)

Scenario: The client brief only explicitly mentions "tracking attendance" and "door access." As an architect, you know there are dozens of *implied* human tasks required to make this actually work in the real world.

Presentation Prompts:

1. **Implied Tasks:** Identify and briefly describe at least 3 critical "User Tasks" that are NOT mentioned in the brief but are absolutely required for the system to function (Hint: Think about what happens when things go wrong).
2. **The "Buddy" Problem:** The brief mentions stopping students from signing for friends. With a swipe card system, what is the new physical risk (e.g., "buddy swiping"), and how might you handle it through software or process design?

Group 3: The Quality & Risk Consultants

Scenario: You are responsible for ensuring the system survives contact with the real world during the 5-minute rush between classes. **Presentation Prompts:**

1. **Quality Attributes:** Identify 3 specific Non-Functional Requirements (Quality Attributes) that are critical for this specific system.
2. **Verification:** Pick one of your quality attributes. How exactly would you verify/test it before the university pays you for the software?
3. **Hidden Concerns:** What is one major security, privacy, or environmental concern you have about this system that the university hasn't thought of yet?

Group 4: The Extensibility Architects (Applying Guy Steele)

Scenario: In his paper "*Growing a Language*", Guy Steele argues you shouldn't design a massive system all at once, nor a tiny one that can't adapt. You must design a system that acts as a "pattern for growth."

Presentation Prompts:

1. **Primitives to Complex:** In Steele's paper, he starts with 1-syllable primitives to build complex words. In our system, what is the lowest-level data "primitive" generated by the hardware? How do you combine those primitives to create a complex domain concept like an `UnauthorisedAccessAttempt`?
2. **"Going Meta":** Steele states: "*We should now think of a language design as a tool for making more tools.*" How do you design the core of this attendance system so that next year, the Library IT team can use your core to build a "Book Checkout" tool without having to rewrite your code?

Group 5: The System Philosophers (Cathedrals vs. Bazaars)

Scenario: Swinburne management is debating how to build this system. The central IT department wants to build a "Cathedral" (a massive, rigid, top-down master plan for the whole university). The student developer club wants to build a "Bazaar" (open APIs where any department can hack together what they need).

Presentation Prompts:

1. **The Cathedral Risk:** According to Steele and Alexander (quoted in the paper), what is the primary danger of Swinburne imposing a strict "Master Plan / Cathedral" architecture on this system from Day 1?
2. **The Shopping Mall Solution:** Steele ultimately suggests a "Shopping Mall" approach (a compromise between the Cathedral and Bazaar). What does a "Shopping Mall" software architecture look like for a university campus? Who builds what?